

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 5, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 What you get
 - 2.2 Two paths
 - 2.2.1 Try it with Docker (recommended)
 - 2.2.2 Develop locally
 - 2.3 Distribution
 - 2.4 License
- 3 Run with Docker images
- 4 Run with Docker images
 - 4.1 Images (GHCR)
 - 4.2 Option A — Ansible deploy (edge VM)
 - 4.3 Option B — Local trial (compose on one machine)
 - 4.4 Required configuration
 - 4.5 Start / stop
 - 4.6 Next
- 5 First login and health check
- 6 First login and health check
 - 6.1 Health endpoints
 - 6.2 Login
 - 6.3 Smoke checklist
 - 6.4 Troubleshooting
- 7 Quick Start
- 8 Quick Start
- 9 Updating the stack
- 10 Updating the stack
 - 10.1 Image-only upgrade (preserve historian)
 - 10.2 Full redeploy
 - 10.3 Verify after upgrade
 - 10.4 Rollback
- 11 Local development
- 12 Local development
 - 12.1 Clone and auth
 - 12.2 Start stack
 - 12.3 BACnet bench overlay (optional)
 - 12.4 Docs preview
- 13 Build Docker images

14	Build Docker images
14.1	Local build
14.2	Publish to GHCR (maintainers)
15	Developer Guide
16	Developer Guide
17	Tests and CI
18	Tests and CI
18.1	Local tests
18.2	CI (GitHub Actions)
18.3	Docs build (same as CI)
19	Contributing
20	Contributing
20.1	Expectations
20.2	Pull request checklist
20.3	Repository layout (short)
20.4	Security issues
20.5	Agent-assisted development
21	System overview
22	System overview
22.1	Components
22.2	Optional PyPI-only path
23	Data flow
24	Data flow
24.1	Telemetry ingest
24.2	Rule evaluation
24.3	Model sync
24.4	Retention
25	Containers
26	Containers
27	Architecture
28	Architecture
29	Dashboard overview
30	Dashboard overview
30.1	Primary areas
30.2	Live updates
30.3	Roles
31	Auth and roles
32	Auth and roles
32.1	Production (LAN / edge)
32.2	Localhost dev
33	Rule Lab
34	Rule Lab
34.1	Contract
34.2	Storage
34.3	Workflow
35	Model workflow
36	Model workflow
36.1	Brick model
36.2	Typical integrator flow
37	Operator Bridge
38	Operator Bridge
39	Network setup

40	Network setup
40.1	Bind address
40.2	Discovery range
40.3	Verify
41	Discover and read
42	Discover and read
42.1	Operator workflow
43	Polling
44	Polling
44.1	Enable on edge
44.2	Health
45	Write safety
46	Write safety
46.1	Defaults
46.2	Enable writes (commission role)
46.3	Operator rules
47	BACnet
48	BACnet
49	Python recipes
50	Python recipes
50.1	1 — Flatline (1 hour window)
50.2	2 — Spread / delta (1 hour)
50.3	3 — Out-of-bounds (rolling avg)
50.4	4 — Rate of change (per hour)
50.5	5 — Deadband / hysteresis
50.6	6 — Setpoint vs measured
50.7	7 — Occupancy schedule gating
50.8	8 — Consecutive true samples
50.9	9 — Mark entire lookback window
50.10	10 — Missing data
50.11	Promotion checklist
51	YAML recipes
52	YAML recipes
52.1	Minimal rule file
52.2	When to use YAML vs Python Rule Lab
53	Windowing and debugging
54	Windowing and debugging
54.1	Rolling average
54.2	Window helpers
54.3	Debug print
54.4	Trace mode
54.5	Common false positives
54.6	Unit conversion
55	Rule Cookbook
56	Rule Cookbook
56.1	Rule anatomy (Python)
57	Convention
58	Fault code convention
58.1	Format
58.2	Categories
58.3	Severity
58.4	Linking rules

58.5 Cookbook mapping	
59 AHU faults	
60 AHU faults	
61 VAV faults	
62 VAV faults	
63 RTU faults	
64 RTU faults	
65 Sensor and data quality	
66 Sensor and data quality	
66.1 Communication quality	
66.2 Alarm console quality	
67 Fault Codes	
68 Fault Codes	
69 LAN hardening	
70 LAN hardening	
70.1 Deployment modes	
70.2 Network exposure	
70.3 Rule Lab caution	
71 Secrets and auth	
72 Secrets and auth	
72.1 Files (gitignored)	
72.2 Required variables (production)	
72.3 GHCR pulls	
72.4 Backup	
73 BACnet write allowlists	
74 BACnet write allowlists	
75 Security	
76 Security	
77 API routes	
78 API routes	
78.1 Health & audit	
78.2 Auth	
78.3 Rules & FDD	
78.4 Rule Lab playground	
78.5 BRICK / data model	
78.6 BACnet	
78.7 Faults (check-engine)	
78.8 Timeseries & sites	
78.9 Building & host	
78.10 Local agent (Ollama)	
78.11 Modbus	
78.12 PyPI engine API (separate)	
78.13 Errors	
79 Python package	
80 Python package (open-fdd)	
80.1 Modules	
80.2 When to use package-only	
80.3 Versioning	
80.4 Offline example	
81 Configuration reference	
82 Configuration reference	
82.1 Bridge / auth	

82.2	BACnet
82.3	Docker edge
83	Glossary
84	Glossary
85	Appendix
86	Appendix
87	Arrow-native FDD runtime
88	Arrow-native FDD runtime
88.1	Rule contract
88.2	Legacy row rules
88.3	Threading
88.4	Benchmarks

Generated June 5, 2026

1 Home

2 Open-FDD

Open-FDD is an open-source platform for **fault detection**, **BACnet integration**, and **operator analytics** at the building edge. It combines a React dashboard, FastAPI Operator Bridge, Python Rule Lab, local feather historian, and optional Brick/RDF modeling.

Who it is for

Audience	Start here
IT / controls engineer trying the stack	Quick Start — Docker
Developer contributing or customizing	Developer Guide
Integrator wiring APIs	Appendix — API routes
Analyst authoring FDD rules	Rule Cookbook · Fault Codes

2.1 What you get

Component	Purpose
Operator Bridge	Web API + dashboard (trends, faults, Rule Lab, model tools)

Component	Purpose
BACnet tools	Discover, read, poll, optional supervised writes
Rule Lab	Test Python <code>evaluate(row, cfg, ...)</code> rules on live or historical data
Local historian	Feather-based telemetry store on the edge host
Brick / RDF model	Equipment and point semantics for bindings and analytics
Docker images	Published on GHCR — pull and run without building from source
Python package	<code>pip install open-fdd</code> for offline CSV/notebook and YAML engine use

2.2 Two paths

2.2.1 Try it with Docker (recommended)

Pull published images from `ghcr.io/bbartling/`, configure auth and BACnet env files, start the stack, open the dashboard.

→ [Quick Start](#)

2.2.2 Develop locally

Clone the repo, create `workspace/auth.env.local`, build images, run tests, submit a PR.

→ [Developer Guide](#)

2.3 Distribution

Channel	Use when
GHCR images <code>ghcr.io/bbartling/openfdd-*</code>	Production or trial edge deploy
This repository	Custom builds, BACnet commissioning, Rule Lab development
PyPI <code>open-fdd</code>	

Channel	Use when
	Library-only pandas/YAML workflows (no full UI)

2.4 License

MIT — see repository LICENSE.

3 Run with Docker images

4 Run with Docker images

4.1 Images (GHCR)

Image	Role
ghcr.io/bbartling/openfdd-bridge	FastAPI + React dashboard
ghcr.io/bbartling/openfdd-commission	BACnet discover / read / write agent
ghcr.io/bbartling/openfdd-bacnet-poll	Historian poll driver (host network)
ghcr.io/bbartling/openfdd-mcp-rag	Optional doc-search sidecar

Check [GitHub Packages](#) for current tags (e.g. 2026.06.07-edge).

4.2 Option A — Ansible deploy (edge VM)

From your **control machine** (laptop or CI runner with SSH to the edge host):

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd

# One-time: build dashboard assets if not in the image tag you use
./scripts/build_operator_dashboard.sh prod

# Set image tag from a published GHCR release
export OPENFDD_IMAGE_TAG=2026.06.07-edge

# Deploy (requires inventory + host_vars – see infra/ansible/inventory.example.yml)
```

```
cd infra/ansible
export SSHPASS='your-ssh-password' # or use SSH keys
./deploy.sh docker --limit <inventory_host>
```

Host vars should set `openfdd_docker_pull_from_ghcr: true`. Site BACnet bind, model, and rules sync from `infra/ansible` templates and `edge_backup/local/<site>/`.

Bootstrap helper (docker → Caddy → optional AI → ops check):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/
bootstrap_edge_ghcr.sh --limit <inventory_host>
```

4.3 Option B — Local trial (compose on one machine)

For a quick lab trial without Ansible:

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local # edit
passwords
./scripts/docker_build.sh # or pull GHCR tags into compose via
env
./scripts/openfdd_stack.sh up
```

Open `http://127.0.0.1:8765/` (bridge) or `http://127.0.0.1/` if Caddy is installed locally.

4.4 Required configuration

File	Purpose
<code>workspace/auth.env.local</code>	Operator / integrator login and <code>OFDD_AUTH_SECRET</code>
<code>workspace/bacnet/commissioning/commission.env</code>	BACnet bind IP, instance, discover range
<code>workspace/data/</code>	Bind-mounted historian, rules, model (created on first run)

Copy from `*.example` files in the repo. Never commit real passwords.

4.5 Start / stop

```
# Ansible edge: re-run deploy or on host:
cd ~/open-fdd/docker && docker compose up -d
```



```
# Local dev:
./scripts/openfdd_stack.sh up
./scripts/openfdd_stack.sh down
```

4.6 Next

→ First login and health check

5 First login and health check

6 First login and health check

6.1 Health endpoints

URL	Auth	Meaning
GET /health	Public	Bridge process alive
GET /health/ stack	Bearer token	Container traffic-light summary

```
curl -s http://<host>:8765/health | jq .
# or through Caddy:
curl -s http://<host>/health | jq .
```

Stack script (from repo on control machine or edge):

```
./scripts/stack_health_check.sh <edge-ip-or-hostname>
```

6.2 Login

1. Open the dashboard URL (Caddy :80 → bridge :8765).
2. Sign in with credentials from workspace/auth.env.local (default roles: integrator, operator).
3. Confirm **Building insight** / home loads without API errors.

```
curl -s -X POST http://<host>/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"<from-auth-env>"}
```

6.3 Smoke checklist

Check	Pass criteria
Dashboard HTML	200 on /
Auth	POST /api/auth/login returns token
Model	GET /api/model/health shows point count
BACnet	Commission container running; discover returns devices (if OT LAN attached)
Historian	Feather store growing under workspace/data/feather_store/
Rule Lab	Lint + test-rule APIs respond (integrator role)

6.4 Troubleshooting

Symptom	Likely fix
401 on all routes	Set <code>auth.env.local</code> ; disable <code>OFDD_AUTH_DISABLED</code> on LAN
Empty BACnet discover	Fix <code>BACNET_BIND</code> in <code>commission.env</code> — see BACnet network setup
Caddy shows welcome page	Reload Caddy config pointing to bridge :8765
Stale UI after upgrade	Hard refresh; confirm new bridge image tag deployed

7 Quick Start

8 Quick Start

Run Open-FDD on a Linux edge host or VM using **published Docker images**. No local image build required.

Step	Page
1	Run with Docker images
2	First login and health check
3	Updating the stack

Prerequisites: Linux host with Docker and Docker Compose, network access to BACnet devices if polling OT equipment, and a safe lab or maintenance window for first BACnet discovery.

BACnet writes are disabled by default. Do not enable write paths on live equipment without an allowlist and operator authorization. See [Write safety](#).

9 Updating the stack

10 Updating the stack

10.1 Image-only upgrade (preserve historian)

Pull a new GHCR tag **without** wiping feather data or BACnet config on the edge host:

```
export OPENFDD_IMAGE_TAG=<new-tag>
RUN_POST_CHECK=1 ./scripts/upgrade_edge_ghcr.sh --limit
<inventory_host>
```

This sets `openfdd_docker_sync_workspace_data=false` so bind-mounted `~/open-fdd/workspace/data/` is not replaced.

10.2 Full redeploy

When you need to sync site model, rules, or commission files from Ansible backups:

```
OPENFDD_IMAGE_TAG=<tag> ./deploy.sh docker --limit
<inventory_host>
```

10.3 Verify after upgrade

```
./scripts/stack_health_check.sh <host>
ls -la ~/open-fdd/workspace/data/feather_store/
# on edge – size should not drop to zero
docker compose -f ~/open-fdd/docker/docker-compose.yml ps
```

10.4 Rollback

Re-deploy the previous known-good OPENFDD_IMAGE_TAG. Workspace data on the host survives image-only upgrades; keep backups of workspace/data/ before major config changes.

11 Local development

12 Local development

12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

12.2 Start stack

```
./scripts/build_and_test.sh          # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

URL	Service
http://127.0.0.1:8765	Bridge (direct)
http://127.0.0.1:8765/docs	OpenAPI

12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env – set BACNET_BIND to your OT interface
```

12.4 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve
# http://127.0.0.1:4000/open-fdd/
```

13 Build Docker images

14 Build Docker images

14.1 Local build

```
./scripts/docker_build.sh  
# Optional tar for air-gap:  
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-bacnet-poll, openfdd-mcp-rag.

14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch with a version tag.

Maintainer checklist:

1. Green CI on master
2. Run workflow with tag e.g. 2026.06.06-edge
3. Verify packages at ghcr.io/bbartling/openfdd-*
4. Deploy to a demo edge with `OPENFDD_IMAGE_TAG=<tag>`

Details live in `.github/workflows/` and `scripts/docker_build.sh` — not duplicated here.

15 Developer Guide

16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

Topic	Page
Local setup	Local development
Docker builds	Build Docker images
Tests & CI	Tests and CI

Topic	Page
PRs	Contributing

AI-assisted contributors: see repository `AGENTS.md` for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

17 Tests and CI

18 Tests and CI

18.1 Local tests

```
./scripts/build_and_test.sh # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q # PyPI package tests
cd workspace/dashboard && npm test # dashboard unit tests (if
configured)
```

18.2 CI (GitHub Actions)

Workflow	Trigger	Checks
ci.yml	PR / push	pytest, bridge security audit, dashboard build, Jekyll docs
docs-pages.yml	push master	GitHub Pages site
publish-open-fdd.yml	tag open-fdd-v*	PyPI wheel
publish-docker-addons.yml	manual	GHCR images

18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches docs/.

19 Contributing

20 Contributing

20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (`auth.env.local`, Ansible secrets/`*.env.local`, commissioning CSVs with real site data).
- Follow existing code style in `workspace/api/` and `workspace/dashboard/`.

20.2 Pull request checklist

☐

Tests pass locally (`./scripts/build_and_test.sh` or `scoped pytest`)

☐

Docs updated if user-facing behavior changed

☐

`docs/` Jekyll build succeeds

☐

No credentials in diff

20.3 Repository layout (short)

Path	Contents
<code>workspace/api/</code>	Operator Bridge (FastAPI)
<code>workspace/dashboard/</code>	React SPA
<code>open_fdd/</code>	PyPI package (engine, playground, reports)
<code>docker/</code>	Compose files and image contexts
<code>infra/ansible/</code>	Edge deploy playbooks
<code>docs/</code>	This site (Jekyll + Just the Docs)

20.4 Security issues

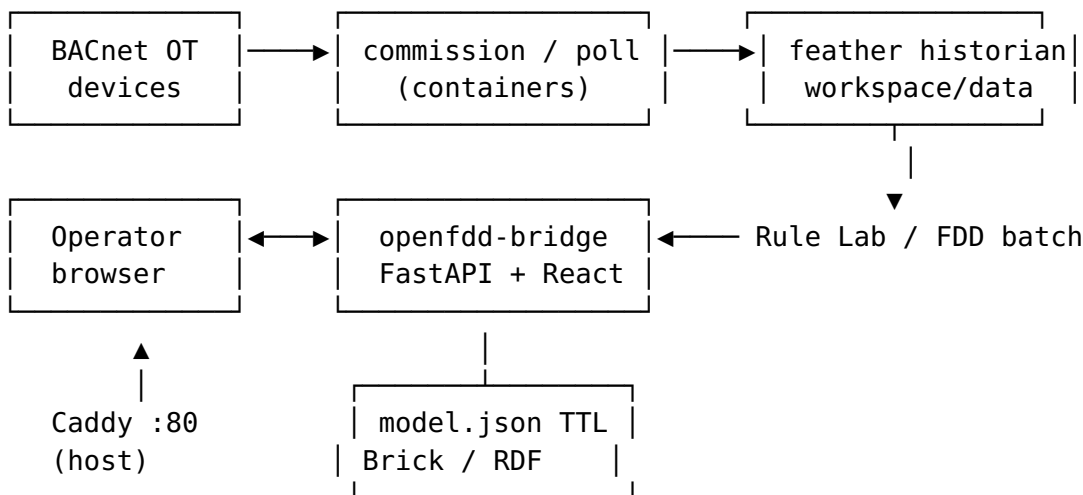
Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

20.5 Agent-assisted development

Cursor/Claude contributors: read AGENTS.md and skills/ for automation boundaries. Public docs intentionally avoid agent playbook detail.

21 System overview

22 System overview



22.1 Components

Layer	Responsibility
Operator Bridge	Auth, REST/WebSocket API, compiled dashboard, Rule Lab
BACnet commission	Who-Is, read property, supervised writes (gated)
BACnet poll	Scheduled RPM reads → historian
Historian	Feather files per point; local-first retention
FDD runner	Executes workspace/data/rules_py/*.py on schedules or API batch

Layer	Responsibility
Model	Brick TTL + model.json bindings (fdd_input, equipment tree)
MCP RAG	Optional local doc retrieval for agent tools
Caddy	Host reverse proxy :80 → bridge (typical edge)
Ollama	Optional host or container LLM for building insight (not required for FDD)

22.2 Optional PyPI-only path

`pip install "open-fdd[engine]"` runs the YAML/pandas engine; `pip install open-fdd` covers the playground sandbox **without** the Operator Bridge UI. Use for notebooks, CSV backfills, or cloud lambdas. See [Appendix — Python package](#).

23 Data flow

24 Data flow

24.1 Telemetry ingest

1. **Poll driver** reads BACnet points on a schedule (host-network container).
2. Samples land in the **feather store** (workspace/data/feather_store/).
3. Point registry (points.csv, discovered inventory) ties BACnet objects to series_id.

24.2 Rule evaluation

1. Rules are Python files in workspace/data/rules_py/ with metadata in rules_store.json.
2. **Bindings** map rules to points via the Brick model (fdd_input tags).
3. Batch runner (POST /api/rules/batch or host timer) produces fdd_results.json.
4. Dashboard **faults** view and check-engine status read aggregated results.

24.3 Model sync

- Commissioning imports update model.json / TTL graph.
- POST /api/model/sync-ttl refreshes SPARQL-backed tree used by the UI.

24.4 Retention

Feather files grow on disk; retention policies are configurable on the edge. Image upgrades should **not** delete workspace/data/ when using upgrade_edge_ghcr.sh.

25 Containers

26 Containers

Container	Network	Persistent mounts
openfdd-bridge	bridge	workspace/ (data, api static, auth env)
openfdd-commission	host or bridge	workspace/bacnet/
openfdd-bacnet-poll	host	workspace/data/, poll profiles
openfdd-mcp-rag	bridge	read-only docs index

Host services (typical edge): Caddy, optional Ollama, openfdd-fdd-loop.timer (docker exec batch).

State lives on the **host filesystem** under workspace/ — containers are replaceable. Prefer GHCR image upgrades over re-provisioning the VM when possible.

27 Architecture

28 Architecture

High-level view of the Open-FDD edge stack.

Page	Topic
System overview	Components and responsibilities
Data flow	Telemetry → historian → FDD → dashboard
Containers	Docker services and host boundaries

29 Dashboard overview

30 Dashboard overview

30.1 Primary areas

Area	Purpose
Home / Building insight	Check-engine status (green/yellow/red), active fault summary
Trends	Point history from feather store
Faults	Rule hits grouped by equipment family
Rule Lab	Edit, lint, and test Python rules
Data model	Equipment tree, point bindings, import health
BACnet	Discovery, reads, commission workflows
Settings / health	Stack status, auth mode

30.2 Live updates

Dashboard tiles use REST polling and WS `/ws/dashboard` for selected live views (ticket auth via POST `/api/auth/ws-ticket`).

30.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#).

31 Auth and roles

32 Auth and roles

32.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

Role	Typical access
viewer	Read trends, faults, model tree
operator	Run batch FDD, acknowledge views
integrator	Rule Lab, bindings, model import
commission	BACnet writes (plus write guard)
agent	Agent tools / app-edit gates
admin	Full configuration

Login: `POST /api/auth/login` → Bearer JWT on API calls.

32.2 Localhost dev

`OFDD_BRIDGE_HOST=127.0.0.1` with `OFDD_AUTH_DISABLED=1` is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: [Security](#).

33 Rule Lab

34 Rule Lab

Rule Lab is the in-browser editor for Python FDD rules.

34.1 Contract

```
def evaluate(row, cfg, prev_row=None, rows=None):  
    """Return False, True, or (True, window_rows) for retroactive  
    paint."""
```

34.2 Storage

Artifact	Path
Rule source	workspace/data/rules_py/<id>.py
Metadata	workspace/data/rules_store.json
Bindings	Model fdd_input + POST /api/rules/bind

Saving in the UI or via API writes both metadata and .py file.

34.3 Workflow

1. **Lint** — POST /api/playground/lint
2. **Test** — POST /api/playground/test-rule on a building time window
3. **Bind** — attach rule to a point in the model
4. **Promote** — enable rule and run batch FDD

Recipes and patterns: [Rule Cookbook](#). Tag rules with fault codes from [Fault Codes](#).

35 Model workflow

36 Model workflow

36.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (workspace/data/data_model.ttl)
- **Point registry** — BACnet inventory → series_id for historian bindings
- **Health score** — orphan points, missing fdd_input, stale telemetry

36.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, ...) in the model UI.
3. Tag points with fdd_input roles (zone temp, SAT, damper cmd, ...).
4. Bind Rule Lab rules to those inputs.
5. Run `P0ST /api/model/sync-ttl` after bulk imports.

Export: `GET /api/model/export` (TTL/JSON). API detail in [Appendix](#).

37 Operator Bridge

38 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

Page	Topic
Dashboard overview	Main UI areas
Auth and roles	Login and permissions
Rule Lab	Python rule authoring
Model workflow	Brick bindings and imports

REST details: [Appendix — API routes](#).

39 Network setup

40 Network setup

40.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses `network_mode`: `host` on Linux edges.

40.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

40.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

41 Discover and read

42 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

Capability	Notes
Who-Is / I-Am	Device discovery on OT subnet
Read property	Present value, object name, units
RPM	Bulk reads for inventory export
Write property	Gated — see Write safety

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

42.1 Operator workflow

1. Confirm [network setup](#).

2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

43 Polling

44 Polling

The **bacnet-poll** container runs scheduled reads defined in `device_poll_profiles.csv` and enabled points in `points.csv`.

Setting	Location
Poll profiles	workspace/bacnet/polls/
Enabled points	workspace/data/points.csv
Output	workspace/data/feather_store/

44.1 Enable on edge

Host var `enable_bacnet_poll_driver: true` in Ansible (or compose profile). Driver requires host networking on Linux.

44.2 Health

- Dashboard poll status shows last cycle time.
- Stale points trigger data-quality fault patterns — see [Sensor quality faults](#).

45 Write safety

46 Write safety

BACnet writes can change live plant behavior. Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

46.1 Defaults

Control	Default
Supervisory writes	Off
Write allowlist	Empty — must enumerate device/object
Audit	Commands logged when writes enabled

46.2 Enable writes (commission role)

1. Set env flag documented in [Security](#) (OFDD_BACNET_WRITE_ENABLED or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.
4. Test on bench hardware before production OT.

46.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

47 BACnet

48 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

Page	Topic
Network setup	Bind address and NIC
Discover and read	Commission agent
Polling	Historian driver
Write safety	Defaults and allowlists

49 Python recipes

50 Python recipes

Each recipe: **use case** → **inputs** → **code** → **behavior** → **tuning** → **false positives**.

50.1 1 — Flatline (1 hour window)

Use case: Stuck sensor, failed thermistor, frozen BACnet value.

Inputs: temp or rh in row; rows history ≥ 1 h span.

```
from open_fdd.playground.cookbook import cfg_threshold,
hour_window_ready, window_rows_1h

def evaluate(row, cfg, prev_row=None, rows=None):
    if not rows:
        return False
    window = window_rows_1h(row, rows)
    if not hour_window_ready(window):
        return False
    vals = [r.get("temp") for r in window if r.get("temp") is not
None]
    if len(vals) < 2:
        return False
    spread = max(vals) - min(vals)
    tol = cfg_threshold(cfg, "flatline_tolerance")
    if spread < tol:
        return True, window
    return False
```

Tuning | Start with flatline_tolerance 0.05-0.15 °F for zone temp |
False positives | Very tight spaces with real stable temp; increase
tolerance or require longer window |

Fault codes: VAV-C, AHU-C, BLD-B — [Fault Codes](#).

50.2 2 — Spread / delta (1 hour)

Use case: Poor delta-T, hunting, capacity issue.

```
from open_fdd.playground.cookbook import cfg_threshold,
hour_window_ready, window_rows_1h
```

```

def evaluate(row, cfg, prev_row=None, rows=None):
    window = window_rows_1h(row, rows or [])
    if not hour_window_ready(window):
        return False
    vals = [r["temp"] for r in window if r.get("temp") is not
None]

    if len(vals) < 2:
        return False
    return (max(vals) - min(vals)) > cfg_threshold(cfg,
"max_spread")

```

Tuning | max_spread 2-6 °F depending on equipment |
False positives | Morning warm-up, large OAT swing |

50.3 3 — Out-of-bounds (rolling avg)

Use case: Comfort or equipment limit violations.

```

from open_fdd.playground.cookbook import cfg_threshold

def evaluate(row, cfg, prev_row=None, rows=None):
    v = row.get("temp_rolling_avg") or row.get("temp")
    if v is None:
        return False
    return v < cfg_threshold(cfg, "bounds_low") or v >
cfg_threshold(cfg, "bounds_high")

```

Tuning | Set bounds per zone type; use rolling avg to debounce spikes |
False positives | Unoccupied setback if schedule not gated — see recipe 7 |

50.4 4 — Rate of change (per hour)

Use case: Runaway heating/cooling, open door, coil valve stuck open.

```

def evaluate(row, cfg, prev_row=None, rows=None):
    if not prev_row or prev_row.get("temp") is None or
row.get("temp") is None:
        return False
    dt_h = (row["ts_ms"] - prev_row["ts_ms"]) / 3_600_000
    if dt_h <= 0:
        return False
    rate = abs(row["temp"] - prev_row["temp"]) / dt_h
    return rate > float(cfg.get("max_temp_per_hour", 5))

```

False positives | Single bad sample — pair with rolling avg or consecutive hits |

50.5 5 — Deadband / hysteresis

Use case: Chattering alarm near setpoint.

```
def evaluate(row, cfg, prev_row=None, rows=None):
    sp = float(cfg.get("setpoint", 72))
    band = float(cfg.get("deadband", 2))
    v = row.get("temp")
    if v is None:
        return False
    if prev_row and prev_row.get("_latched"):
        # clear inside tighter band
        return abs(v - sp) > band * 1.5
    return abs(v - sp) > band
```

Store latch in row dict only for retroactive paint; for production use cfg-backed state via consecutive sample recipe.

50.6 6 — Setpoint vs measured

Use case: VAV discharge temp far from SAT setpoint.

```
def evaluate(row, cfg, prev_row=None, rows=None, series=None):
    if not series:
        return False
    sat_sp = series.get("SAT_SP", {}).get("current")
    sat = row.get("temp")
    if sat_sp is None or sat is None:
        return False
    return abs(sat - sat_sp) > float(cfg.get("max_deviation", 3))
```

Requires series_aliases in rule config for cross-point rules.

50.7 7 — Occupancy schedule gating

Use case: Only evaluate during occupied hours.

```
def evaluate(row, cfg, prev_row=None, rows=None):
    if not cfg.get("occupied", True): # set via binding metadata
    or static cfg
        return False
```

```
# ... inner fault logic ...  
return False
```

Prefer binding-level enable flags or schedule tables in `cfg`.

50.8 8 — Consecutive true samples

Use case: Debounce transient spikes.

```
def evaluate(row, cfg, prev_row=None, rows=None):  
    need = int(cfg.get("consecutive", 3))  
    if not rows or row["row"] < need - 1:  
        return False  
    for r in rows[row["row"] - need + 1 : row["row"] + 1]:  
        if not _inner(r, cfg):  
            return False  
    return True  
  
def _inner(r, cfg):  
    v = r.get("temp")  
    return v is not None and v > float(cfg.get("high", 80))
```

50.9 9 — Mark entire lookback window

Return `(True, window_rows)` so the sweep paints every row in the window (retroactive mode).

50.10 10 — Missing data

```
def evaluate(row, cfg, prev_row=None, rows=None):  
    if row.get("temp") is None or row.get("quality") == "bad":  
        return True  
    return False
```

Pair with comms fault codes — Sensor quality.

50.11 Promotion checklist

☐

Lint clean in Rule Lab

☐

Tested on ≥ 48 h of real building data

- ☐ Thresholds documented in rule config
- ☐ Fault code assigned (VAV-C, etc.)
- ☐ False positive review with operator
- ☐ Enabled only after binding to correct fdd_input

51 YAML recipes

52 YAML recipes

For **offline** pandas workflows with `open_fdd.engine.RuleRunner` (no Operator Bridge).

```
pip install "open-fdd[engine]"
```

52.1 Minimal rule file

```
id: zone_high_temp
type: threshold
column: zone_temp
params:
  upper: 80
  lower: 65

from open_fdd.engine import RuleRunner
import pandas as pd

df = pd.read_csv("telemetry.csv")
runner = RuleRunner.from_yaml("rules/zone_high_temp.yaml")
flags = runner.run(df)
```

52.2 When to use YAML vs Python Rule Lab

YAML engine	Python Rule Lab
CSV/backfill notebooks	Live edge + dashboard
Fixed rule types in engine	Arbitrary <code>evaluate()</code> logic

YAML engine	Python Rule Lab
CI on historical extracts	BACnet-bound production rules

Rule type reference and column maps: [Appendix — Python package](#).

53 Windowing and debugging

54 Windowing and debugging

54.1 Rolling average

Rule Lab enriches rows with `temp_rolling_avg` / `degF_rolling_avg` before `evaluate()` (default 1, 5, or 10 minutes by `ts_ms`).

Set `rolling_avg_minutes` in rule config or test API body.

54.2 Window helpers

Helper	Purpose
<code>window_rows_1h(row, rows)</code>	Samples in trailing 60 minutes
<code>hour_window_ready(window)</code>	True when window spans $\geq 95\%$ of 1 h

54.3 Debug print

`print()` inside `evaluate()` appears in Rule Lab event console **on fault hits** (or verbose test mode). Use sparingly in production rules.

54.4 Trace mode

Enable **verbose** on test-rule API to see window spread diagnostics without firing the rule on every row.

54.5 Common false positives

Symptom	Mitigation
Warm-up transient	Gate on occupancy or minimum window fill
Poll dropout	Require <code>samples_in_avg</code> minimum
Unit mismatch	Set <code>temp_unit</code> in <code>cfg</code> ; see <code>open_fdd.playground.temp_units</code>
Single spike	Rolling avg + consecutive sample recipe

54.6 Unit conversion

Rules use `row["temp"]` in the rule's configured unit (imperial °F default). MQTT/historian rows include both `degF` and `degC`.

55 Rule Cookbook

56 Rule Cookbook

Practical patterns for **Python Rule Lab** (`evaluate(row, cfg, ...)`) and optional **YAML** engine rules (`pip install "open-fdd[engine]"`).

Page	Audience
Python recipes	Operator Bridge Rule Lab
YAML recipes	Offline pandas / PyPI engine
Windowing and debugging	Tuning and false positives

56.1 Rule anatomy (Python)

```
def evaluate(row, cfg, prev_row=None, rows=None):
    # row: current sample (temp, ts_ms, rolling avg fields, ...)
    # cfg: thresholds from rule config in UI
    # rows: full history in test window – use for lookbacks
    return False # or True, or (True, window_rows)
```

Import helpers from `open_fdd.playground.cookbook` in production rules, or use site `bench_fdd_common` shims where present.

57 Convention

58 Fault code convention

58.1 Format

FAMILY-SUFFIX — suffix is **1-3 letters** (e.g. VAV-C, AHU-B). Numeric suffixes like VAV-03 are avoided (collision with physical equipment names).

58.2 Categories

Category	Meaning
performance_degradation	Efficiency or capacity drift
simultaneous_heat_cool	Heating and cooling fighting
sensor_fault	Flatline, OOB, inconsistent readings
io_fault	Command vs feedback mismatch

58.3 Severity

Level	Operator response
info	Log; trend for maintenance window
warning	Investigate within days
critical	Prompt attention; may affect comfort/energy significantly

58.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into `GET /api/faults/status` by family.

58.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`) — see [Rule Cookbook](#).

59 AHU faults

60 AHU faults

Air handling unit codes (sample from catalog).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-A	Supply fan performance degradation	warning	Spread/runtime vs baseline	Belt slip, dirty filters, VFD limit	Trend speed vs airflow; filter dP
AHU-B	Simultaneous heating and cooling	critical	Heat + cool outputs active together	Sequencing bug, leaking valve	Trend coil commands; valve feedback
AHU-C	SAT sensor fault	warning	Flatline / OOB SAT	Failed sensor, bad calibration	SAT trend; compare to coil discharge
AHU-D	MAT inconsistent with OAT/RAT	warning	MAT outside OAT-RAT envelope	MAT/OAT/RAT sensor errors	Verify damper positions; cross-check OAT
AHU-E	Economizer not economizing	warning	Free cool available, OA minimum	Stuck damper, lockout	OA damper vs OAT trend
AHU-F	Damper command vs feedback mismatch	warning	Cmd $\neq$ feedback	Stuck actuator, linkage	Compare cmd/feedback trends

Example rule: flatline_1h on SAT $\rightarrow$ tag **AHU-C**. Recipe: [Python recipes](#).

61 VAV faults

62 VAV faults

Variable air volume terminal codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
VAV-A	Zone heating/cooling overlap	warning	Heat and cool at terminal	Reheat valve stuck, SAT too low	Zone temp vs SAT; valve cmd
VAV-B	Poor zone temperature control	warning	Persistent deviation from setpoint	Oversized box, bad tuning	PI loop trends; occupancy
VAV-C	Zone temperature sensor fault	warning	Flatline / OOB zone temp	Failed zone sensor	Flatline test; compare to neighbor zones
VAV-D	Minimum airflow not met	warning	Flow below minimum cmd	Damper stuck, pressure issue	Airflow vs damper cmd
VAV-E	Discharge temp deviation from SAT	warning	DAT far from SAT setpoint	Coil issue, sensor drift	DAT vs SAT SP

Example rule: oob_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

63 RTU faults

64 RTU faults

Packaged rooftop unit codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-A	Compressor short cycling	warning	Rapid on/off cycles	Low charge, oversized unit	Run-time histogram
RTU-B	Economizer / OA damper fault	warning	OA behavior vs conditions	Actuator, sensor	OA damper vs enthalpy/OAT
RTU-C	Supply air temperature fault	warning	SAT flatline or OOB	Sensor failure	SAT trend vs outdoor conditions
RTU-D	Simultaneous heat and cool	critical	Heat + cool stages together	Control board, relay stuck	Stage status trends

Bind rules to packaged unit fdd_input points (SAT, OAT, compressor cmd, ...).

65 Sensor and data quality

66 Sensor and data quality

Code	Title	Severity	Detection summary	Likely causes	Operator checks
BLD-B	Outdoor air sensor fault	warning	OAT flatline / unrealistic	Sensor exposure, failed probe	Compare to weather service
BLD-D	Stale telemetry	warning	No new samples within threshold	Poll driver down, device offline	Poll health; BACnet comms
DC-C	Datacom sensor anomaly	warning	CRAH/ room sensor flatline	Failed sensor	Row-level flatline recipe

66.1 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use `stale_points` cookbook pattern for dropout detection.
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

66.2 Alarm console quality

When integrating with BMS alarms (future/external), map Open-FDD codes to operator work orders — Open-FDD itself is **analytics-first**, not a dial-out alarm server.

67 Fault Codes

68 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

Page	Content
Convention	Naming, severity, categories
AHU faults	Air handling units
VAV faults	Terminal units
RTU faults	Packaged rooftops
Sensor & data quality	Stale, flatline, comms

Live catalog API: GET `/api/faults/catalog` on the Operator Bridge.

69 LAN hardening

70 LAN hardening

70.1 Deployment modes

Mode	Bind	Auth
Local dev	127.0.0.1	Optional OFDD_AUTH_DISABLED=1
Lab LAN (insecure)	0.0.0.0	Requires explicit insecure dev flags — not production
Edge production	0.0.0.0 behind Caddy	Required credentials + OFDD_AUTH_SECRET

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are explicitly set).

70.2 Network exposure

- Do not port-forward the bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; bridge management on IT VLAN with firewall rules.
- Prefer Caddy on :80/443 rather than exposing :8765 directly.

70.3 Rule Lab caution

Rule Lab executes **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

71 Secrets and auth

72 Secrets and auth

72.1 Files (gitignored)

File	Contents
workspace/ auth.env.local	JWT secret, role passwords
infra/ansible/secrets/ <host>.env.local	SSH deploy secrets (maintainers)

Copy from *.example templates only.

72.2 Required variables (production)

Variable	Purpose
OFDD_AUTH_SECRET	HMAC signing (32+ chars)
OFDD_INTEGRATOR_USER / PASSWORD	Rule Lab, bindings
OFDD_OPERATOR_USER / PASSWORD	Operations

72.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

72.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

73 BACnet write allowlists

74 BACnet write allowlists

Supervisory writes require:

- 1. **Feature flag** enabled in environment (off by default).
- 2. **Allowlist** file at workspace/bacnet/write_allowlist.json (copy from [write_allowlist.example.json](#)). If the flag is on but the allowlist is missing, writes are **denied**.
- 3. **Integrator** role on API calls.
- 4. **Audit** log review after commissioning sessions.

Allowlist entries support device_instance, object_identifier, property_identifier, optional priority_min/priority_max, value_min/value_max, and allowed_values for binary/multistate targets.

Lab-only override: OFDD_BACNET_WRITE_ALLOW_ANY=1 (audited; not for production edges).

See operator guide: [BACnet write safety](#).

Never enable blanket write access for agent or integrator automation without human approval per site.

75 Security

76 Security

Concise deployment security for the Operator Bridge and BACnet tools.

Page	Topic
LAN hardening	Auth and bind modes
Secrets and auth	Env files
BACnet write allowlists	Supervisory write gates

77 API routes

78 API routes

REST API served by **openfdd-bridge** (default `http://127.0.0.1:8765`).
Production: Caddy on `:80` proxies to the bridge.

OpenAPI: GET `/docs` and GET `/redoc` when the bridge runs.

Auth: JWT via POST `/api/auth/login`. Most routes require
Authorization: Bearer `<token>`. Roles: viewer, operator, integrator,
commission, agent, admin. BACnet writes require commission role +
write safety. Dev: `OFDD_AUTH_DISABLED` on loopback only.

WebSocket: WS `/ws/dashboard` — POST `/api/auth/ws-ticket`, then
pass the ticket via Sec-WebSocket-Protocol: `ofdd.ws, <ticket>`.
Query `?ticket=` is dev-only when `OFDD_WS_ALLOW_QUERY_TICKET=1`.
Tickets are short-lived and not interchangeable with the Bearer
token.

78.1 Health & audit

Method	Path	Role	Description
GET	<code>/health</code>	public	Minimal liveness (ok, service, version, auth_required)
GET	<code>/health/ stack</code>	auth	Stack traffic-light (verbose URLs/bind with <code>OFDD_DEBUG_DIAGNOSTICS</code>)
POST	<code>/api/auth/ ws-ticket</code>	auth	Short-lived WebSocket ticket
GET	<code>/api/ audit/ summary</code>	auth	Audit counters
GET	<code>/api/ audit/ events</code>	auth	Audit log (query params)
GET	<code>/api/ audit/ errors</code>	auth	Recent API errors

*Exact public set depends on `OFDD_AUTH_DISABLED` and middleware.

78.2 Auth

Method	Path	Description
POST	/api/auth/login	Username/password → JWT
GET	/api/auth/me	Current user
GET	/api/auth/status	Auth mode / logout hints

78.3 Rules & FDD

Method	Path	Role	Description
GET	/api/rules/saved	user	Rule index
POST	/api/rules/save	operator+	Create/update rule metadata
GET	/api/rules/saved/{id}/source	user	Python source
PUT	/api/rules/saved/{id}/source	operator+	Write .py file
GET	/api/rules/assignments	user	Point bindings
POST	/api/rules/bind	integrator+	Bind rule to point
POST	/api/rules/bindings	integrator+	Bulk bindings
POST	/api/rules/batch	operator+	Run all enabled rules → fdd_results.json
GET/POST	/api/rules/drafts	user	Draft rules

78.4 Rule Lab playground

Method	Path	Description
POST	/api/playground/lint	AST lint Python rule
POST	/api/playground/test-rule	Run evaluate() on feather window

Method	Path	Description
POST	/api/playground/run-script	Execute script mode
GET	/api/playground/sample-frame	Sample DataFrame for editor

78.5 BRICK / data model

Method	Path	Description
GET	/api/model/sites	Site list
GET	/api/model/tree	Equipment tree (SPARQL)
GET	/api/model/graph	RDF graph fragment
GET	/api/model/scope	Scoped query
GET	/api/model/export	Export TTL/JSON
GET	/api/model/health	Model point/equipment counts
GET	/api/model/ttl	TTL metadata
POST	/api/model/sync-ttl	Regenerate TTL from JSON
POST	/api/model/import	Import model payload
POST	/api/model/sites	Create site
GET/ POST	/api/model/bacnet-sync	BACnet ↔ model sync state

78.6 BACnet

Method	Path	Role	Description
GET	/config/bacnet	read	Commission URL / config
GET	/api/bacnet/commission/status	read	Commission agent health
GET	/api/bacnet/server/points	read	Server point table
GET		read	Discovered inventory

Method	Path	Role	Description
	/api/ bacnet/ inventory		
POST	/api/ bacnet/ whois	commission	Who-Is
POST	/api/ bacnet/ discover	commission	Discover devices
POST	/api/ bacnet/read	commission	Read property
POST	/api/ bacnet/ read- multiple	commission	RPM read
POST	/api/ bacnet/ write	write	Supervisory write (gated)
POST	/api/ bacnet/ import-to- model	integrator	Inventory → BRICK
POST	/api/ bacnet/ driver/ sync- discovery	commission	Sync driver tree
POST	/api/ bacnet/ poll/once	commission	Trigger poll cycle
GET	/api/ bacnet/ poll/status	read	Poll worker status
POST	/ingest/ bacnet	read	Ingest samples into feather

Bind address: BACNET_BIND in commission env — see [BACnet network setup](#). Operator guide: [Discover and read](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |
 DELETE | /api/bacnet/driver/point/{point_id} | commission | Remove point from registry |

DELETE | /api/bacnet/driver/device/{device_instance} |
commission | Remove device |
DELETE | /api/bacnet/driver/registry | commission | Clear CSVs +
model sync |

78.7 Faults (check-engine)

Method	Path	Description
GET	/api/faults/catalog	Fixed fault codes
GET	/api/faults/status	GREEN/YELLOW/RED aggregate
GET	/api/faults/tree	Fault tree by equipment
GET	/api/faults/code/ {code}	Code metadata

78.8 Timeseries & sites

Method	Path	Description
GET	/api/timeseries/sites	Feather sites
GET	/api/timeseries/series	Series metadata
GET	/api/timeseries/plot	Plot payload
GET	/api/timeseries/readings	Raw readings
GET	/api/sites	Site list
GET	/api/sites/{site_id}/frame	Wide frame for site

78.9 Building & host

Method	Path	Description
GET	/api/building/status	Building summary
GET	/api/building/alerts	Active alerts
GET	/api/host/stats	CPU/mem/disk

78.10 Local agent (Ollama)

Prefix `/openfdd-agent`. Role `agent` for tool execution.

Method	Path	Description
GET	/openfdd-agent/context	Composed prompt context
GET	/openfdd-agent/tools	Tool manifest
POST	/openfdd-agent/tool	Execute tool (e.g. rules.save)
GET	/openfdd-agent/ollama/health	Ollama reachability
GET	/openfdd-agent/building-insight	Check-engine narrative context
GET	/openfdd-agent/zone-temps	Zone temperature insight
POST	/openfdd-agent/chat	Chat completion (catalog-bound)

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview](#).

78.11 Modbus

Method	Path	Description
POST	/api/modbus/read_registers	Read holding registers

78.12 PyPI engine API (separate)

Library-only HTTP is **not** bundled. See [Python package](#) for RuleRunner and open_fdd.reports.

78.13 Errors

JSON error bodies; stack traces hidden unless OFDD_DEBUG_TRACEBACKS=1. See [LAN hardening](#).

79 Python package

80 Python package (open-fdd)

Install from PyPI:

```
pip install "open-fdd[engine]"    # YAML RuleRunner + pandas
pip install open-fdd              # playground sandbox only
```

80.1 Modules

Module	Purpose
open_fdd.engine	YAML RuleRunner on pandas DataFrames
open_fdd.playground	evaluate() sandbox (Rule Lab parity, AWS lambda)
open_fdd.reports	Optional summary/plot helpers

80.2 When to use package-only

Scenario	Use
Notebook on CSV export	engine + YAML rules
Cloud lambda (no UI)	playground.rule_lab
Full building operator stack	Docker Operator Bridge

80.3 Versioning

Publish: git tag open-fdd-vX.Y.Z → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

80.4 Offline example

```
from open_fdd.engine import RuleRunner
import pandas as pd

df = pd.read_csv("ahu_telemetry.csv")
```

```
runner = RuleRunner.from_yaml("rules/high_sat.yaml")
result = runner.run(df)
```

API surface: open_fdd/engine docstrings and docs/
config_schema.json for YAML rule shapes.

81 Configuration reference

82 Configuration reference

82.1 Bridge / auth

Variable	Default	Notes
OFDD_BRIDGE_HOST	127.0.0.1	0.0.0.0 for LAN
OFDD_AUTH_SECRET	—	Required for LAN
OFDD_AUTH_DISABLED	0	Localhost dev only
OFDD_INSECURE_LAN_DEV	0	Lab only with auth disabled

82.2 BACnet

Variable	Notes
BACNET_BIND	ip/mask:47808 in commission.env
OFDD_BACNET_WRITE_ENABLED	Default off — site-specific name may vary

82.3 Docker edge

Variable	Notes
OPENFDD_IMAGE_TAG	GHCR tag for Ansible deploy
openfdd_docker_pull_from_ghcr	Ansible host_var

Full schema for YAML engine rules: docs/config_schema.json.

83 Glossary

84 Glossary

Term	Definition
Operator Bridge	FastAPI service serving API + static dashboard
Rule Lab	UI and APIs for Python <code>evaluate()</code> rules
Feather store	On-disk columnar historian under <code>workspace/data/feather_store/</code>
fdd_input	Brick tag linking a point to a rule binding role
Check-engine	Building-level GREEN/YELLOW/RED fault summary
Commission	BACnet discovery/read (and optional write) agent container
Poll driver	Scheduled BACnet RPM reads into historian
GHCR	GitHub Container Registry — ghcr.io/bbartling/openfdd- *

85 Appendix

86 Appendix

Reference material for integrators and library users.

Page	Content
API routes	Operator Bridge REST/WebSocket
Python package	PyPI <code>open-fdd</code>
Configuration reference	Key env variables
Glossary	Terms

87 Arrow-native FDD runtime

88 Arrow-native FDD runtime

Open-FDD 3.x runs rule-based HVAC fault detection on an **Arrow-native columnar execution engine** by default.

Feather / Arrow IPC

- Arrow Table / RecordBatch stream
- `apply_faults_arrow(table, cfg, context)`
- Boolean fault mask
- summaries / events (JSON for UI)

88.1 Rule contract

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Rules return a `pyarrow.BooleanArray` or `ChunkedArray` with the same row count as the input table.

88.2 Legacy row rules

Old `evaluate(row, cfg, ...)` rules are supported only when:

- the rule sets `"backend": "legacy_row"`, or
- `OPEN_FDD_FDD_BACKEND=legacy_row` is set.

The Rule Lab shows a migration message for legacy rules.

88.3 Threading

Variable	Purpose
<code>OPEN_FDD_ARROW_THREADS</code>	Arrow CPU thread pool
<code>OPEN_FDD_ARROW_IO_THREADS</code>	Arrow I/O threads (when supported)
<code>OPEN_FDD_ARROW_BATCH_ROWS</code>	RecordBatch chunk size (default 50000)
<code>OPEN_FDD_ARROW_PARALLEL_RULES</code>	Parallel rule batches

Variable	Purpose
OPEN_FDD_ARROW_PARALLEL_SITES	Parallel site jobs

Large jobs use Arrow projection/filtering, chunked batches, and job-level parallelism across sites/rules. Not every scalar kernel uses every core; throughput comes from batch scans plus parallel orchestration.

88.4 Benchmarks

```
python benchmarks/bench_arrow_native.py --rows 10000 100000 --
repeats 3 --output .bench/arrow_native.json
python benchmarks/bench_arrow_end_to_end.py --sites 1 --rows-per-
site 100000 --repeats 2 --output .bench/arrow_end_to_end.json
```